Memoria del Proyecto: ShinyVillage

Asignatura: Desarrollo de Videojuegos
Alumno: [Tu nombre]
Fecha: Marzo 2026
Repositorio: [URL del repositorio GitHub]

Índice

1. Introducción
2. Planificación
3. Análisis y Diseño
4. Implementación
5. Pruebas
6. Conclusiones
7. Referencias

---

1. Introducción

1.1 Descripción del proyecto

ShinyVillage es un videojuego RPG 2D de vista cenital desarrollado con **Unity** y **C#**. El jugador controla un personaje que puede moverse por un mapa basado en *tilemaps*, interactuar con el entorno, recoger objetos y gestionarlos mediante un sistema de inventario.

El proyecto se ha desarrollado como práctica de programación orientada a objetos aplicada al desarrollo de videojuegos, explorando patrones de diseño habituales en la industria como el **Singleton**, el uso de **ScriptableObjects** para datos de items y la separación entre lógica y presentación (UI).

1.2 Objetivos

- Implementar un sistema de movimiento 2D con animaciones direccionales.
- Crear un sistema de inventario con slots apilables.
- Desarrollar la interacción con tiles del mapa mediante *Tilemaps* de Unity.
- Construir una interfaz de usuario funcional para el inventario.
- Aplicar el patrón Singleton para el gestor global del juego.

1.3 Tecnologías utilizadas

Tecnología	Versión	Uso
Unity	6 (URP)	Motor de juego principal
C#	—	Lenguaje de programación
Universal Render Pipeline (URP)	17.3.0	Renderizado 2D con iluminación

Tecnología	Versión	Uso
Unity Input System	1.18.0	Gestión de entradas del jugador
TextMeshPro	—	Textos en la UI del inventario
Git / GitHub	—	Control de versiones

2. Planificación

2.1 Metodología

El desarrollo se ha organizado en **fases iterativas**, añadiendo funcionalidad de forma incremental y probando cada sistema antes de pasar al siguiente. Las fases han sido:

1. Movimiento del jugador y animaciones
2. Cámara de seguimiento
3. Sistema de items y recolección
4. Sistema de inventario (lógica)
5. UI del inventario
6. Interacción con el Tilemap
7. GameManager y gestión global

2.2 Control de versiones

El proyecto está alojado en **GitHub**. Se ha trabajado en la rama principal (`main`), con commits frecuentes que documentan el progreso de cada funcionalidad.

3. Análisis y Diseño

3.1 Requisitos funcionales

- **RF01** — El jugador puede moverse en 8 direcciones con animaciones acordes.
- **RF02** — El jugador puede recoger items al colisionar con ellos en el mapa.
- **RF03** — Los items recogidos se almacenan en un inventario de 21 slots.
- **RF04** — Los items del mismo tipo se apilan en el mismo slot (hasta 99 por slot).
- **RF05** — El jugador puede abrir y cerrar el inventario con la tecla `Tab`.
- **RF06** — El jugador puede soltar items del inventario al mapa.
- **RF07** — El jugador puede interactuar con tiles marcados como interactuables pulsando `Espacio`.
- **RF08** — La cámara sigue al jugador en todo momento.

3.2 Arquitectura del proyecto

El proyecto sigue una arquitectura de **componentes** propia de Unity, donde cada `MonoBehaviour` tiene una responsabilidad clara:

```
GameManager (Singleton)
├── ItemManager      → Diccionario de items disponibles
└── TileManager      → Gestión del Tilemap interactuable

Player
├── Movement         → Movimiento físico y animaciones
├── Inventory        → Lógica del inventario (slots)
└── Collectable      → Detección de items en el mapa

UI
├── Inventory_UI     → Control del panel de inventario
└── Slot_UI          → Representación visual de cada slot
```

3.3 Diagrama de clases (simplificado)

```
Inventory
  └── List<Slot>
        └── itemName: string
        └── count: int
        └── icon: Sprite

Item ──────────── ItemData (ScriptableObject)
  └── rb2d            └── itemName: string
  └── data            └── icon: Sprite

GameManager ─── ItemManager
  (Singleton)  └── Dictionary<string, Item>
             ─── TileManager
                └── Tilemap interactableMap
```

---

4. Implementación

4.1 GameManager — Patrón Singleton

El `GameManager` centraliza el acceso a los sistemas principales del juego. Usa el patrón **Singleton** para garantizar que sólo existe una instancia y que es accesible desde cualquier script.

```
public class GameManager : MonoBehaviour
{
    // Instancia estática: garantiza una única existencia global
    public static GameManager instance;

    public ItemManager itemManager;
    public TileManager tileManager;

    private void Awake()
```

```
    {
        // Si ya existe otra instancia, destruimos este duplicado
        if (instance != null && instance != this)
        {
            Destroy(this.gameObject);
        }
        else
        {
            instance = this; // Nos convertimos en la instancia única
        }

        // El GameManager persiste entre escenas
        DontDestroyOnLoad(this.gameObject);

        // Obtenemos los managers desde los componentes del mismo GameObject
        itemManager = GetComponent<ItemManager>();
        tileManager = GetComponent<TileManager>();
    }
}
```

¿Por qué Singleton aquí? Porque el GameManager necesita ser accedido por muchos sistemas (Player, UI, Collectable...) sin necesidad de referencias directas en el Inspector.

4.2 Movimiento del jugador

El script `Movement` captura la entrada del usuario y mueve al jugador. Se separa `Update` (lectura de input) de `FixedUpdate` (física) para evitar el efecto de "rebote" con los colliders.

```
public class Movement : MonoBehaviour
{
    public float speed;          // Velocidad configurable desde el Inspector
    public Animator animator;    // Referencia al Animator del sprite

    private Vector3 direction;   // Dirección actual del movimiento

    private void Update()
    {
        // Leemos los ejes de entrada (valores -1, 0 o 1)
        float horizontal = Input.GetAxisRaw("Horizontal");
        float vertical   = Input.GetAxisRaw("Vertical");

        // Combinamos en un vector y normalizamos para que la diagonal
        // no sea más rápida que los movimientos cardinales
        direction = new Vector3(horizontal, vertical).normalized;

        animateMovement(direction);
    }

    private void FixedUpdate()
```

```
    {
        // Movemos el transform en FixedUpdate para sincronizarlo
        // con el motor de física y evitar el efecto rebote
        transform.position += direction * speed * Time.deltaTime;
    }

    void animateMovement(Vector3 direction)
    {
        if (animator != null)
        {
            if (direction.magnitude > 0) // Si hay movimiento
            {
                animator.SetBool("isMoving", true);
                // Pasamos la dirección al Animator para elegir la animación
correcta
                animator.SetFloat("horizontal", direction.x);
                animator.SetFloat("vertical",   direction.y);
            }
            else // Sin movimiento → animación idle
            {
                animator.SetBool("isMoving", false);
            }
        }
    }
}
```

4.3 Sistema de Inventario

El inventario se compone de dos clases: `Inventory` (lógica) y `Slot` (clase interna que representa cada hueco).

```
[System.Serializable] // Permite ver los datos en el Inspector de Unity
public class Inventory
{
    [System.Serializable]
    public class Slot
    {
        public string itemName; // Nombre del item en este slot
        public int count;       // Cantidad de items apilados
        public int maxAllowed;  // Máximo apilable (por defecto 99)
        public Sprite icon;     // Icono para mostrar en la UI

        public Slot()
        {
            itemName   = "";
            count      = 0;
            maxAllowed = 99;
        }
```

```
        // ¿Cabe un item más en este slot?
        public bool CanAddItem() => count < maxAllowed;

        // Añade un item: actualiza nombre, icono y contador
        public void AddItem(Item item)
        {
            this.itemName = item.data.itemName;
            this.icon     = item.data.icon;
            count++;
        }

        // Elimina un item y limpia el slot si queda vacío
        public void RemoveItem()
        {
            if (count > 0)
            {
                count--;
                if (count == 0)
                {
                    icon     = null;
                    itemName = "";
                }
            }
        }
    }

    public List<Slot> slots = new List<Slot>();

    // El constructor crea todos los slots vacíos de una vez
    public Inventory(int numSlots)
    {
        for (int i = 0; i < numSlots; i++)
            slots.Add(new Slot());
    }
}
```

Decisiones de diseño relevantes:

- El inventario **no es un MonoBehaviour**, ya que es pura lógica de datos, sin presencia en la escena.
- Se usa `[System.Serializable]` para poder visualizarlo en el Inspector y depurar fácilmente.
- Los items del mismo tipo se apilan en el mismo slot buscando un slot existente con ese `itemName`.

4.4 Interacción con el Tilemap

`TileManager` gestiona el mapa de tiles interactuables. Al iniciar la escena, todas las tiles del mapa interactuable se sustituyen por una tile invisible, de forma que el jugador no ve las marcas de interacción hasta que actúa sobre ellas.

```
public class TileManager : MonoBehaviour
{
    [SerializeField] private Tilemap interactableMap;       // Mapa con tiles
interactuables
    [SerializeField] private Tile hiddenInteractableTile;   // Tile invisible
que sustituye a las originales
    [SerializeField] private Tile InteractedTile;           // Tile que muestra
que ya fue interactuada

    public void Start()
    {
        // Recorremos todas las posiciones del mapa y las "ocultamos"
        foreach (var position in interactableMap.cellBounds.allPositionsWithin)
        {
            interactableMap.SetTile(position, hiddenInteractableTile);
        }
    }

    // Comprueba si la tile en esa posición es interactuable
    public bool IsInteractable(Vector3Int position)
    {
        TileBase tile = interactableMap.GetTile(position);
        // Solo consideramos interactuable la tile invisible ("invisible_int")
        return tile != null && tile.name == "invisible_int";
    }

    // Marca una tile como "ya interactuada" cambiando su sprite
    public void SetInteracted(Vector3Int position)
    {
        interactableMap.SetTile(position, InteractedTile);
    }

    // Convierte coordenadas de mundo a coordenadas de celda del tilemap
    public Vector3Int GetCellPosition(Vector3 worldPosition)
    {
        return interactableMap.WorldToCell(worldPosition);
    }
}
```

La interacción se activa desde `Player.cs` al pulsar `Espacio`: se calcula la celda frente al jugador según su dirección actual y se comprueba si es interactuable.

4.5 UI del Inventario

La UI se divide en `Inventory_UI` (lógica del panel) y `Slot_UI` (representación de cada slot individual).

```
public class Inventory_UI : MonoBehaviour
{
    public GameObject inventoryPanel; // Panel visual del inventario
```

```
    public Player player;
    public List<Slot_UI> slots = new List<Slot_UI>(); // Lista de slots
visuales

    void Update()
    {
        // Tab abre y cierra el inventario
        if (Input.GetKeyDown(KeyCode.Tab))
            ToggleInventory();
    }

    public void ToggleInventory()
    {
        if (!inventoryPanel.activeSelf)
        {
            inventoryPanel.SetActive(true);
            Refresh(); // Actualizamos la vista al abrir
        }
        else
        {
            inventoryPanel.SetActive(false);
        }
    }

    // Sincroniza los slots visuales con los datos del inventario del jugador
    public void Refresh()
    {
        for (int i = 0; i < slots.Count; i++)
        {
            // Un slot se considera ocupado si tiene nombre Y cantidad > 0
            if (i < player.inventory.slots.Count
                && player.inventory.slots[i].itemName != ""
                && player.inventory.slots[i].count > 0)
            {
                slots[i].SetItem(player.inventory.slots[i]);
            }
            else
            {
                slots[i].SetEmpty(); // Slot vacío → icono transparente
            }
        }
    }

    // Suelta un item al mapa y lo elimina del inventario
    public void Remove(int slotID)
    {
        Item itemToDrop = GameManager.instance.itemManager.GetItemByName(
            player.inventory.slots[slotID].itemName
        );

        if (itemToDrop != null)
        {
            player.DropItem(itemToDrop); // Spawn del item en el mapa
```

```
                player.inventory.Remove(slotID);
                Refresh(); // Actualiza la vista
            }
        }
    }
```

4.6 Cámara

La cámara sigue al jugador manteniendo un *offset* fijo calculado en el `Start`.

```
public class CameraFollow : MonoBehaviour
{
    [SerializeField] private Transform target; // Transform del jugador
    Vector3 camOffset; // Desplazamiento inicial entre cámara y jugador

    void Start()
    {
        // Calculamos el offset una sola vez al inicio
        camOffset = transform.position - target.position;
    }

    private void FixedUpdate()
    {
        // Actualizamos en FixedUpdate para suavizar el movimiento
        // junto con la física del jugador
        transform.position = target.position + camOffset;
    }
}
```

---

5. Pruebas

5.1 Pruebas realizadas

Funcionalidad	Prueba	Resultado
Movimiento	Mover en 8 direcciones	☑ Correcto
Animaciones	Animación idle y en movimiento	☑ Correcto
Recoger item	Colisionar con item en el mapa	☑ Correcto
Inventario lleno	Intentar recoger con 21 slots llenos	☐ Pendiente
Abrir inventario	Pulsar Tab	☑ Correcto
Soltar item	Botón de soltar en slot con item	☑ Correcto
Interacción tile	Pulsar Espacio frente a tile interactuable	☑ Correcto

Funcionalidad	Prueba	Resultado
Persistencia GameManager	Cambio de escena	☐ Pendiente

5.2 Bugs encontrados y soluciones

Bug 1 — Slots mostrando cuadrado blanco vacío
Los slots vacíos mostraban un cuadrado blanco porque Unity asigna un sprite por defecto blanco a los componentes `Image`. La solución fue poner el color alfa a 0 (`new Color(1,1,1,0)`) cuando el slot está vacío en `Slot_UI.SetEmpty()`.

Bug 2 — Efecto rebote al caminar junto a paredes
El movimiento en `Update` provocaba que la física y el movimiento se calcularan en órdenes distintos. La solución fue mover `transform.position` a `FixedUpdate`, que se sincroniza con el motor de física.

Bug 3 — Items recogidos automáticamente al soltarlos
Al soltar un item, el collider del jugador lo detectaba inmediatamente. La solución fue añadir un *offset* aleatorio (`Random.insideUnitCircle * 2f`) al punto de spawn para alejarlo del collider del jugador.

---

6. Conclusiones

6.1 Objetivos cumplidos

Se han implementado con éxito todos los sistemas principales del juego: movimiento con animaciones, recolección de items, inventario con stacking, UI funcional e interacción con el tilemap. El uso de patrones como **Singleton** (GameManager) y la separación entre lógica y presentación han facilitado la mantenibilidad del código.

6.2 Dificultades encontradas

La principal dificultad fue la coordinación entre el sistema de física de Unity y el movimiento del jugador, así como la gestión del estado visual del inventario (sincronización entre datos e UI). También supuso un reto entender el ciclo de vida de los `MonoBehaviour` (`Awake`, `Start`, `Update`, `FixedUpdate`) y usarlo correctamente.

6.3 Posibles mejoras futuras

- Implementar un sistema de guardado y cargado del estado del juego.
- Añadir más tipos de items con efectos (consumibles, equipables).
- Ampliar el sistema de interacción con diálogos o eventos de historia.
- Añadir enemigos con IA básica.
- Implementar múltiples escenas/zonas del mapa.

---

7. Referencias

- Documentación oficial de Unity
- Unity Learn — 2D Game Development

- Patrón Singleton en Unity — Game Programming Patterns
- Universal Render Pipeline — Unity Docs
- Unity Input System